

Chapter Three

Inheritance

Derived Class and Base Class

Let's suppose that we have worked long and hard to make the Counter class operate just the way we want, and we're pleased with the results, except for one thing. We really need a way to decrement the count. Perhaps we're counting people entering a bank, and we want to increment the count when they come in and decrement it when they go out, so that the count represents the number of people in the bank at any moment. We could insert a decrement routine directly into the source code of the Counter class. However, there are several reasons that we might not want to do this. First, the Counter class works very well and has undergone many hours of testing and debugging. If we start fooling around with the source code for Counter, the testing process will need to be carried out again, and of course we may foul something up and spend hours debugging code that worked fine before we modified it.

In some situations, there might be another reason for not modifying the Counter class: We might not have access to its source code, especially if it was distributed as part of a class library. To avoid these problems, we can use inheritance to create a new class based on Counter, without modifying Counter itself. Here's the listing for COUNTER, which includes a new class, CountDn, that adds a decrement operator to the Counter class:

```
#include <iostream>
using namespace std;
class Counter //base class
{
protected: //NOTE: not private
    unsigned int count; //count
public:
    Counter() : count(0) //no-arg constructor
    {}
    Counter(int c) : count(c) //1-arg constructor
    {}
    unsigned int get_count() const //return count
    { return count; }
    Counter operator ++ () //incr count (prefix)
    { return Counter(++count); }
};
class CountDn : public Counter //derived class
{
public:
    Counter operator -- () //decr count (prefix)
    { return Counter(--count); }
};
int main()
```

```
{
CountDn c1; //c1 of class CountDn
cout << "\nc1=" << c1.get_count(); //display c1
++c1; ++c1; ++c1; //increment c1, 3 times
cout << "\nc1=" << c1.get_count(); //display it
--c1; --c1; //decrement c1, twice
cout << "\nc1=" << c1.get_count(); //display it
cout << endl;
return 0;
}
```

The listing starts off with the Counter class, which (with one small exception, which we'll look at later) has not changed since its appearance in COUNTPP3. Notice that, for simplicity, we haven't modelled this program on the POSTFIX program, which incorporated the second overloaded ++ operator to provide postfix notation.

Accessing Base Class Members

An important topic in inheritance is knowing when a member function in the base class can be used by objects of the derived class. This is called accessibility. Let's see how the compiler handles the accessibility issue in the COUNTER example.

Substituting Base Class Constructors

In the main() part of COUNTER we create an object of class CountDn: CountDn c1; This causes c1 to be created as an object of class CountDn and initialized to 0. But wait – how is this possible? There is no constructor in the CountDn class specifier, so what entity carries out the initialization? It turns out that – at least under certain circumstances – if you don't specify a constructor, the derived class will use an appropriate constructor from the base class. In COUNTER there's no constructor in CountDn, so the compiler uses the no-argument constructor from Count.

This flexibility on the part of the compiler – using one function because another isn't available – appears regularly in inheritance situations. Generally, the substitution is what you want, but sometimes it can be unnerving.

Substituting Base Class Member Functions

The object c1 of the CountDn class also uses the operator++() and get_count() functions from the Counter class. The first is used to increment c1:

```
++c1;
```

The second is used to display the count in c1:

```
cout << "\nc1=" << c1.get_count();
```

Again, the compiler, not finding these functions in the class of which c1 is a member, uses member functions from the base class.

The protected Access Specifier

We have increased the functionality of a class without modifying it. Well, almost without modifying it. Let's look at the single change we made to the Counter class. The data in the classes we've looked at so far, including count

in the Counter class in the earlier COUNTPP3 program, have used the private access specifier. In the Counter class in COUNTER, count is given a new specifier: protected. What does this do? Let's first review what we know about the access specifiers private and public. A member function of a class can always access class members, whether they are public or private. But an object declared externally can only invoke (using the dot operator, for example) public members of the class. It's not allowed to use private members. For instance, suppose an object objA is an instance of class A, and function funcA() is a member function of A. Then in main() (or any other function that is not a member of A) the statement objA.funcA(); will not be legal unless funcA() is public. The object objA cannot invoke private members of class A. Private members are, well, private.

This is all we need to know if we don't use inheritance. With inheritance, however, there is a whole raft of additional possibilities. The question that concerns us at the moment is, can member functions of the derived class access members of the base class? In other words, can operator--() in CountDn access count in Counter? The answer is that member functions can access members of the base class if the members are public, or if they are protected. They can't access private members.

We don't want to make count public, since that would allow it to be accessed by any function anywhere in the program and eliminate the advantages of data hiding. A protected member, on the other hand, can be accessed by member functions in its own class or – and here's the key – in any class derived from its own class. It can't be accessed from functions outside these classes, such as main(). This is just what we want.

Overriding Member Functions

You can use member functions in a derived class that override – that is, have the same name as – those in the base class. You might want to do this so that calls in your program work the same way for objects of both base and derived classes. Here's an example that modelled a stack, a simple data storage device called STAKARAY. It allowed you to push integers onto the stack and pop them off. However, STAKARAY had a potential flaw. If you tried to push too many items onto the stack, the program might bomb, since data would be placed in memory beyond the end of the st[] array. Or if you tried to pop too many items, the results would be meaningless, since you would be reading data from memory locations outside the array. To cure these defects we've created a new class, Stack2, derived from Stack. Objects of Stack2 behave in exactly the same way as those of Stack, except that you will be warned if you attempt to push too many items on the stack or if you try to pop an item from an empty stack. Here's the listing for STAKEN:

```
#include <iostream>
using namespace std;
#include <process.h> //for exit()
class Stack
```

```
{
protected: //NOTE: can't be private
enum { MAX = 3 }; //size of stack array
int st[MAX]; //stack: array of integers
int top; //index to top of stack
public:
Stack() //constructor
{ top = -1; }
void push(int var) //put number on stack
{ st[++top] = var; }
int pop() //take number off stack
{ return st[top--]; }
};
class Stack2 : public Stack
{
public:
void push(int var) //put number on stack
{
if(top >= MAX-1) //error if stack full
{ cout << "\nError: stack is full"; exit(1); }
Stack::push(var); //call push() in Stack class
}
int pop() //take number off stack
{
if(top < 0) //error if stack empty
{ cout << "\nError: stack is empty\n"; exit(1); }
return Stack::pop(); //call pop() in Stack class
}
};
int main()
{
Stack2 s1;
s1.push(11); //push some values onto stack
s1.push(22);
s1.push(33);
cout << endl << s1.pop(); //pop some values from stack
cout << endl << s1.pop();
cout << endl << s1.pop();
cout << endl << s1.pop(); //oops, popped one too many...
cout << endl;
return 0; }
```

Scope Resolution with Overridden Functions

How do push() and pop() in Stack2 access push() and pop() in Stack? They use the scope resolution operator, ::, in the statements

```
Stack::push(var);
```

and

```
return Stack::pop();
```

These statements specify that the `push()` and `pop()` functions in `Stack` are to be called. Without the scope resolution operator, the compiler would think the `push()` and `pop()` functions in `Stack2` were calling themselves, which – in this case – would lead to program failure. Using the scope resolution operator allows you to specify exactly what class the function is a member of.

Class Hierarchies

The following example program starts with a base class `employee`. This class handles the employee's last name and employee number. From this class three other classes are derived: `manager`, `scientist`, and `labourer`. The `manager` and `scientist` classes contain additional information about these categories of employee, and member functions to handle this information.

Source code:

```
#include <iostream>
using namespace std;
const int LEN = 80; //maximum length of names
class employee //employee class
{
private:
char name[LEN]; //employee name
unsigned long number; //employee number
public:
void getdata()
{
cout << "\n Enter last name: "; cin >> name;
cout << " Enter number: "; cin >> number;
}
void putdata() const
{
cout << "\n Name: " << name;
cout << "\n Number: " << number;
}
};
class manager : public employee //management class
{
private:
char title[LEN]; // "vice-president" etc.
double dues; //golf club dues
public:
void getdata()
{
employee::getdata();
cout << " Enter title: "; cin >> title;
```

```
cout << " Enter golf club dues: "; cin >> dues;
}
void putdata() const
{
    employee::putdata();
    cout << "\n Title: " << title;
    cout << "\n Golf club dues: " << dues;
}
};
class scientist : public employee //scientist class
{
private:
    int pubs; // number of publications
public:
    void getdata()
    {
        employee::getdata();
        cout << " Enter number of pubs: "; cin >> pubs;
    }
    void putdata() const
    {
        employee::putdata();
        cout << "\n Number of publications: " << pubs;
    }
};
class laborer : public employee //laborer class
{
};
int main()
{
    manager m1, m2;
    scientist s1;
    laborer l1;
    cout << endl; //get data for several employees
    cout << "\nEnter data for manager 1";
    m1.getdata();
    cout << "\nEnter data for manager 2";
    m2.getdata();
    cout << "\nEnter data for scientist 1";
    s1.getdata();
    cout << "\nEnter data for laborer 1";
    l1.getdata();
    //display data for several employees
    cout << "\nData on manager 1";
    m1.putdata();
```

```
cout << "\nData on manager 2";  
m2.putdata();  
cout << "\nData on scientist 1";  
s1.putdata();  
cout << "\nData on laborer 1";  
l1.putdata();  
cout << endl;  
return 0;  
}
```

The main() part of the program declares four objects of different classes: two managers, a scientist, and a laborer. (Of course many more employees of each type could be defined, but the output would become rather large.) It then calls the getdata() member functions to obtain information about each employee, and the putdata() function to display this information.

Here's a sample interaction with EMPLOY. First the user supplies the data.

Enter data for manager 1

Enter last name: Wainsworth

Enter number: 10

Enter title: President

Enter golf club dues: 1000000

Enter data on manager 2

Enter last name: Bradley

Enter number: 124

Enter title: Vice-President

Enter golf club dues: 500000

Enter data for scientist 1

Enter last name: Hauptman-Frenglish

Enter number: 234234

Enter number of pubs: 999

Enter data for laborer 1

Enter last name: Jones

Enter number: 6546544

The program then plays it back.

Data on manager 1

Name: Wainsworth

Number: 10

Title: President

Golf club dues: 1000000

Data on manager 2

Name: Bradley

Number: 124

Title: Vice-President

Golf club dues: 500000

Data on scientist 1

Name: Hauptman-Frenglish

Number: 234234

Number of publications: 999

Data on laborer 1

Name: Jones

Number: 6546544

A more sophisticated program would use an array or some other container to arrange the data

so that a large number of employee objects could be accommodated.